Universidad San Carlos de Guatemala
Centro Universitario de Occidente
Ingeniería en Ciencias y Sistemas
Laboratorio de Sistemas de Bases de datos 1
ING. Bryan René Gómez Gómez



Ejercicio #10

API con NestJS y TypeORM

Domingo, 18 de octubre, 2026
Quetzaltenango

Juan Carlos Rodríguez López
202231970

1. Introducción

En este documento presento el Ejercicio #10 del laboratorio. Hice una API con NestJS y TypeORM sobre la base de datos gimnasio, que es la misma que armé en la práctica #7. La API corre en el puerto 3070, que sale de sumar 3000 con los dos últimos dígitos de mi carné (202231970).

La base de datos está en un contenedor de Docker (lab_DB1) y se conecta por el puerto 5435. En la configuración de TypeORM dejé synchronize en true y logging en true, así en la consola de la API aparece cada SQL que genera TypeORM.

2. Entidades y relaciones

El proyecto tiene 5 entidades, una por cada tabla: tipo_membresia, entrenador, socio, clase e inscripcion. Las llaves foráneas las mapeé con @JoinColumn usando el nombre real de la columna en la base. Las relaciones son estas:

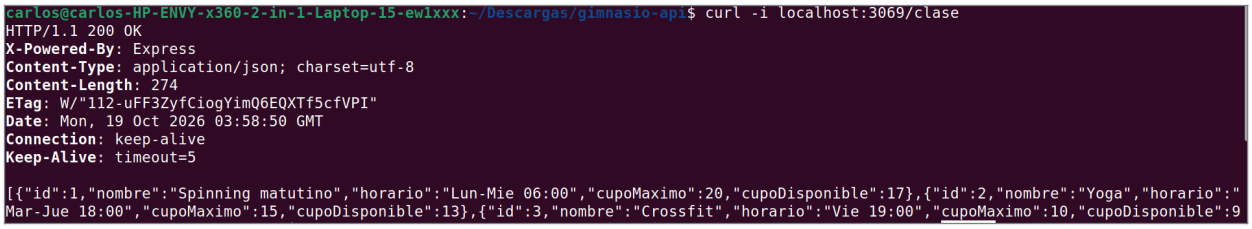
Relación	**Descripción**
TipoMembresia 1:N Socio	Un tipo de membresía tiene muchos socios
Entrenador 1:N Clase	Un entrenador dirige muchas clases
Socio 1:N Inscripcion	Un socio puede tener varias inscripciones
Clase 1:N Inscripcion	Una clase tiene varias inscripciones
Socio M:N Clase	Se resuelve con la tabla inscripcion

El CRUD completo lo hice con la entidad Clase. Además hice dos consultas (una con relations y otra con QueryBuilder) y una transacción para inscribir a un socio en una clase.

3. Pruebas de los endpoints

Todas las pruebas las hice con curl contra localhost, usando la opción -i para ver el código de respuesta. Debajo de cada captura puse el SQL que generó TypeORM para esa petición.

3.1 GET /clase



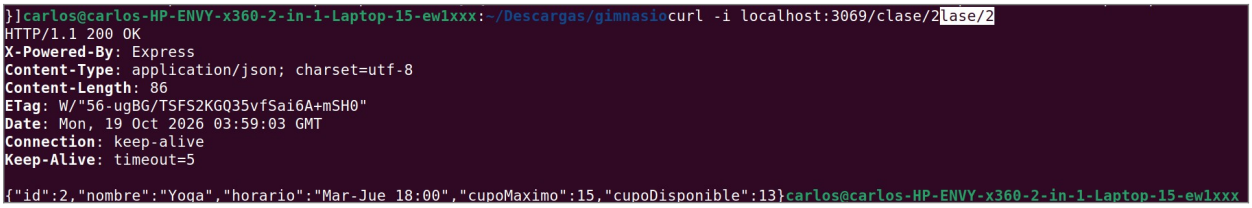
Captura 1. Listar todas las clases

Este endpoint devuelve todas las clases. Responde 200 y trae las 3 clases que ya estaban en la base. TypeORM hace un solo SELECT con ORDER BY id.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
ORDER BY "Clase"."id" ASC
```

3.2 GET /clase/2



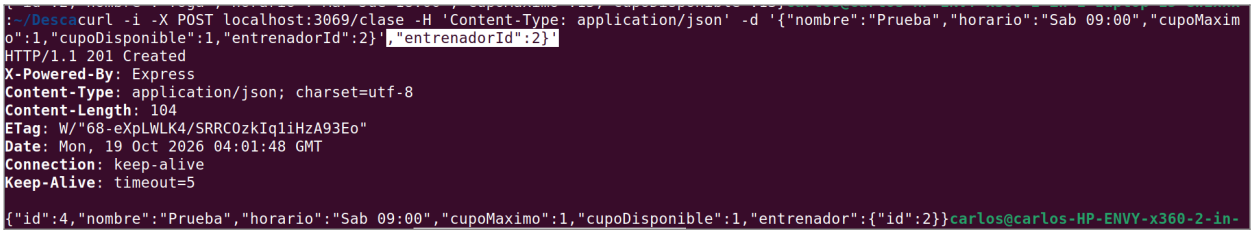
Captura 2. Buscar una clase por id

Busca una clase por su id y devuelve la clase Yoga. El id va como parámetro ($1), no se concatena dentro del SQL, y TypeORM agrega LIMIT 1 porque espera un solo registro.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
WHERE (("Clase"."id" = $1)) LIMIT 1
-- PARAMETERS: [2]
```

3.3 POST /clase



Captura 3. Crear una clase

Con este POST creé una clase llamada Prueba con un solo cupo, porque la voy a usar más adelante para la transacción. Responde 201 y le tocó el id 4. TypeORM hace el INSERT dentro de una transacción, por eso se ven START TRANSACTION y COMMIT.

SQL generado por TypeORM:

```
START TRANSACTION
INSERT INTO "clase"("nombre", "horario", "cupo_maximo", "cupo_disponible", "entrenador_id")
VALUES ($1, $2, $3, $4, $5) RETURNING "id"
-- PARAMETERS: ["Prueba","Sab 09:00",1,1,2]
COMMIT
```

3.4 PATCH /clase/4



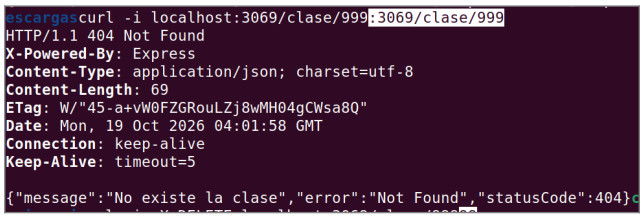
Captura 4. Modificar una clase

Cambié el horario de la clase 4. Primero el servicio busca la clase para revisar que exista y después hace el UPDATE. El UPDATE solo modifica la columna horario porque fue la única que cambió.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
WHERE (("Clase"."id" = $1)) LIMIT 1
-- PARAMETERS: [4]
START TRANSACTION
UPDATE "clase" SET "horario" = $1 WHERE "id" = $2
-- PARAMETERS: ["Sab 10:00",4]
COMMIT
```

3.5 GET /clase/999 (id que no existe)



Captura 5. GET de un id inexistente, responde 404

Como la clase 999 no existe, la API responde 404 Not Found con el mensaje "No existe la clase" y no un 200 vacío. En el log solo aparece el SELECT, que no encontró nada.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
WHERE (("Clase"."id" = $1)) LIMIT 1
-- PARAMETERS: [999]
```

3.6 DELETE /clase/999 (id que no existe)



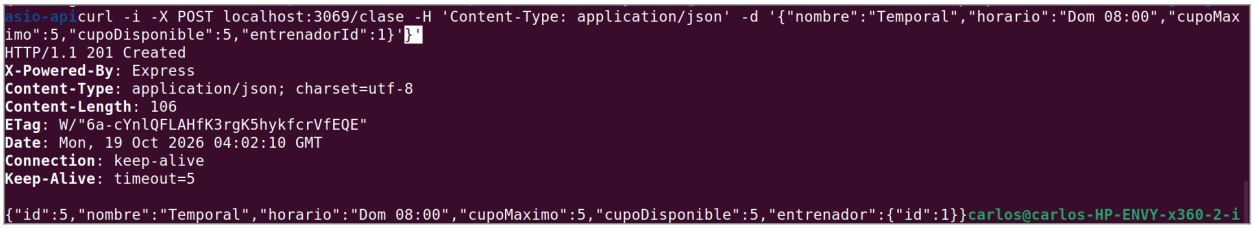
Captura 6. DELETE de un id inexistente, responde 404

Con el DELETE pasa lo mismo. Antes de borrar se busca la clase y, como no existe, responde 404 y nunca se ejecuta un DELETE.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
WHERE (("Clase"."id" = $1)) LIMIT 1
-- PARAMETERS: [999]
```

3.7 POST /clase y DELETE /clase/5 (borrar una clase que sí existe)

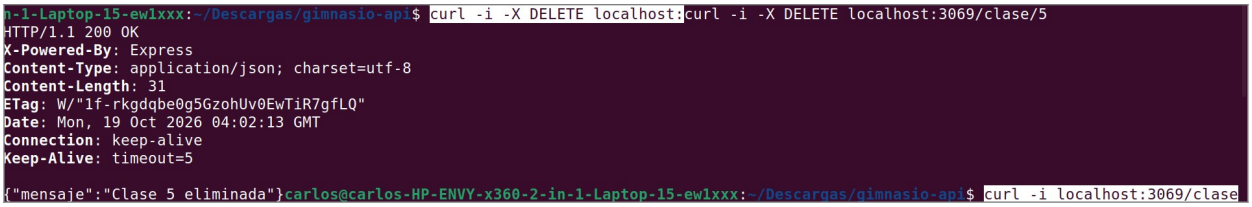


Captura 7. Creación de la clase temporal (id 5)

Para probar el borrado de un registro que sí existe, primero creé una clase temporal. Le tocó el id 5.

SQL generado por TypeORM:

```
START TRANSACTION
INSERT INTO "clase"("nombre", "horario", "cupo_maximo", "cupo_disponible", "entrenador_id")
VALUES ($1, $2, $3, $4, $5) RETURNING "id"
-- PARAMETERS: ["Temporal","Dom 08:00",5,5,1]
COMMIT
```



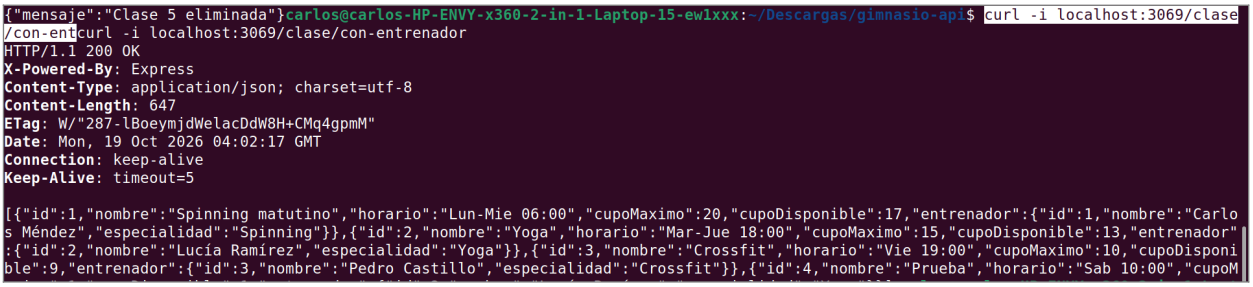
Captura 8. Eliminación de la clase 5

Después la eliminé con DELETE. Responde 200 con el mensaje "Clase 5 eliminada". Se ve el SELECT que revisa que exista y luego el DELETE con el id como parámetro.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ... FROM "clase" "Clase"
WHERE (("Clase"."id" = $1)) LIMIT 1
-- PARAMETERS: [5]
DELETE FROM "clase" WHERE "id" = $1
-- PARAMETERS: [5]
```

3.8 GET /clase/con-entrenador (relations)



Captura 9. Clases con su entrenador

Este endpoint devuelve cada clase junto con su entrenador, usando la opción relations de TypeORM. En el JSON cada clase trae el objeto entrenador con su nombre y su especialidad. El SQL lleva un INNER JOIN entre clase y entrenador. Me salió INNER JOIN y no LEFT JOIN porque la relación es obligatoria, o sea que toda clase tiene un entrenador.

SQL generado por TypeORM:

```
SELECT "Clase"."id" AS "Clase_id", "Clase"."nombre" AS "Clase_nombre", ...,
"Clase__Clase_entrenador"."id" AS "Clase__Clase_entrenador_id", ...
FROM "clase" "Clase"
INNER JOIN "entrenador" "Clase__Clase_entrenador" ON
"Clase__Clase_entrenador"."id"="Clase"."entrenador_id"
ORDER BY "Clase"."id" ASC
```

3.9 QueryBuilder y transacción exitosa



Captura 10. Consulta con QueryBuilder (Yoga) y transacción de inscripción exitosa

En esta captura salen dos peticiones. La primera es GET /clase/especialidad/Yoga, que usa createQueryBuilder. Une clase con entrenador con innerJoinAndSelect, filtra por la especialidad con un parámetro y ordena por el nombre de la clase. Salieron dos clases, Prueba y Yoga, que son las dos del entrenador de yoga. El valor Yoga va como parámetro $1, nunca se concatena.

SQL generado por TypeORM (QueryBuilder):

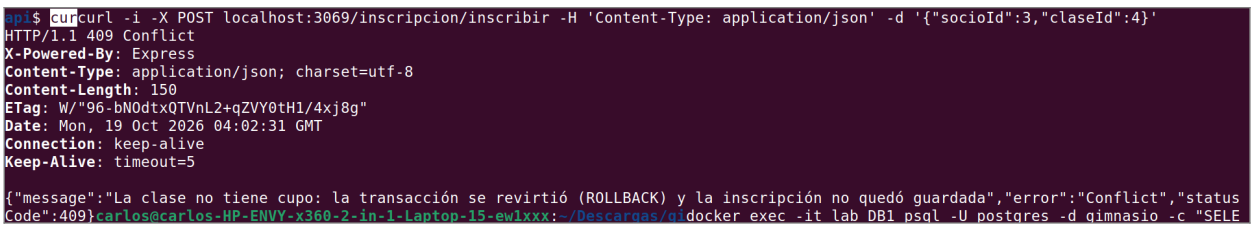
```
SELECT "c"."id" AS "c_id", "c"."nombre" AS "c_nombre", ..., "e"."especialidad" AS
"e_especialidad"
FROM "clase" "c" INNER JOIN "entrenador" "e" ON "e"."id"="c"."entrenador_id"
WHERE "e"."especialidad" = $1 ORDER BY "c"."nombre" ASC
-- PARAMETERS: ["Yoga"]
```

La segunda petición es el POST a /inscripcion/inscribir. Inscribe al socio 4 en la clase 4, que tenía 1 cupo. La transacción hace dos cambios: inserta la inscripción y le resta 1 a cupo_disponible de la clase. Como los dos salieron bien, termina con COMMIT y responde 201.

SQL generado por TypeORM (transacción exitosa):

```
START TRANSACTION
INSERT INTO "inscripcion"("socio_id", "clase_id", "fecha_inscripcion")
VALUES ($1, $2, DEFAULT) RETURNING "id", "fecha_inscripcion"
-- PARAMETERS: [4,4]
UPDATE "clase" SET "cupo_disponible" = "cupo_disponible" - 1 WHERE "id" = $1
-- PARAMETERS: [4]
COMMIT
```

3.10 Transacción que falla (ROLLBACK)

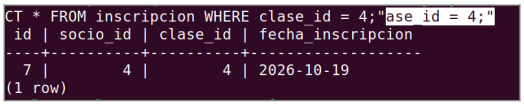


Captura 11. Inscripción en una clase sin cupo, responde 409

Ahora intento inscribir al socio 3 en la misma clase 4, que ya se quedó sin cupo (cupo_disponible = 0). El INSERT en inscripcion sí se ejecuta, pero el UPDATE intenta dejar cupo_disponible en -1 y eso viola el CHECK clase_cupo_disponible_check de la tabla. PostgreSQL manda el error y TypeORM hace ROLLBACK de toda la transacción. La API responde 409 con un mensaje que lo explica.

SQL generado por TypeORM (transacción fallida):

```
START TRANSACTION
INSERT INTO "inscripcion"("socio_id", "clase_id", "fecha_inscripcion")
VALUES ($1, $2, DEFAULT) RETURNING "id", "fecha_inscripcion"
-- PARAMETERS: [3,4]
UPDATE "clase" SET "cupo_disponible" = "cupo_disponible" - 1 WHERE "id" = $1
-- PARAMETERS: [4]
error: new row for relation "clase" violates check constraint "clase_cupo_disponible_check"
ROLLBACK
```



Captura 12. Consulta a la base para comprobar que el INSERT se revirtió

Para comprobar que el primer cambio también se anuló, hice un SELECT a la tabla inscripcion de la clase 4. Solo aparece una fila, la del socio 4. El socio 3 no quedó inscrito, o sea que el INSERT se deshizo junto con el UPDATE que falló.

4. Comparación del SQL del log con SQL escrito a mano

La comparación completa, con el SQL del log y el SQL que escribiría yo a mano lado a lado, está en el archivo comparacion-sql.md que entrego junto con este documento. Lo que más noté fue lo siguiente:

TypeORM le pone alias a todas las columnas (por ejemplo "Clase_id") para poder armar los objetos, y el SQL a mano es más corto y directo. Los valores siempre van como parámetros ($1, $2) y no pegados al texto, que es más seguro. En las consultas con relación usó INNER JOIN porque la llave foránea es obligatoria. Y en las operaciones que guardan datos, TypeORM abre su propia transacción con START TRANSACTION y COMMIT sin que yo lo pida.

5. Conclusión

Con este ejercicio vi cómo un ORM genera el SQL por uno, pero hay que revisarlo porque no siempre es el que uno escribiría. La transacción de inscripción muestra la atomicidad: o se hacen los dos cambios (insertar la inscripción y bajar el cupo) o no se hace ninguno. Cuando la clase no tiene cupo, el CHECK de la tabla hace que falle el UPDATE y se anula también el INSERT.